

Lecture 05: Object Design (Part I)

Responsibility-Driven Design (RDD) — SWE30003 Software Architecture and Design

1. Core Philosophy of Object-Oriented Design (OOD)

OOD is fundamentally distinct from OOP: While Object-Oriented Programming often centers heavily around concrete language configurations, syntax properties, and code deployment variables, high-level structural design mandates a deeper baseline abstraction. A proficient software engineer does not merely write functional source code; they construct a robust, semantic **working vocabulary** compiled explicitly from domain-specific requirements.

The System as a Living Organism: An object-oriented application is architecturally modeled as a dynamic ecosystem of autonomous, cooperating objects. In exact alignment with biological organs inside a living host, every individual software component assumes a highly specialized operational role and interacts systematically across shared boundaries to sustain the execution flow of the broader infrastructure.

2. Fundamental Concepts: Objects, Roles, and Responsibilities

To systematically decompose real-world complexities into clean software layers, Responsibility-Driven Design maps the system footprint using five structural entities:

- **Application:** A cohesive, structured architecture of interacting and collaborating software entities.
- **Object:** A discrete production unit managing private data segments and related methods that serves as a tangible, real-world execution of one or more design roles.
- **Role:** A highly cohesive, logical cluster of closely coupled system responsibilities.
- **Responsibility:** A strict **obligation** mapped to a component to perform an explicit action or maintain specific domain states. These are systematically separated into two functional zones:
 - *Doing Responsibilities:* Actively executing a localized operation, programmatically triggering actions within external components, or orchestrating/coordinating systemic behaviors.
 - *Knowing Responsibilities:* Keeping encapsulated data points private, retaining tracking paths to related external nodes, or maintaining information it can dynamically derive or calculate.
- **Collaboration:** A programmatic interaction mapped across multiple objects or structural roles to satisfy an overarching requirement that fundamentally exceeds the processing boundaries of any single component.

3. The Responsibility-Driven Design Process

The system design lifecycle follows a definitive engineering pipeline, systematically migrating from raw abstract requirements down to production-ready programmatic contracts:

Requirements & Domain Model Analysis



1. Identify System Objects & Roles



2. Distribute & Assign Architectural Responsibilities



3. Formulate Collaborative Interaction Channels



4. Package Logic into Clean Public Interfaces (APIs)

Strategic Rules for Allocation

- **Elimination of Passive "Data Holders":** Engineers must consciously reject the creation of classes designed purely to hold attributes (resembling old procedural structures or basic structs) without associated behaviors. Software entities must possess localized "intelligence" to process and manage their own encapsulated data fields.
- **Decoupling Interface from Implementation:**
 - *Interface:* Formally establishes exactly *what* a component can execute (the immutable public contract exposed to consumer classes).
 - *Implementation:* Strictly conceals precisely *how* the entity handles that contract under the hood. Violating this separation limits system flexibility and breaks consumer modules whenever inner backend logic is modified.

4. Architectural Optimization: Coupling and Cohesion

Cohesion and coupling serve as the definitive architectural metrics used to measure structural health, ensuring the application stays stable, maintainable, and highly portable across changing infrastructure horizons over time:

Design Dimension	Target Standard	Definition & Implementation Strategy
Cohesion	High Cohesion	A single software module or class component must isolate its focus onto **one unambiguous responsibility** with highly integrated behaviors. This is accomplished by grouping internal methods that serve an identical business logic purpose.
Coupling	Low Coupling	Minimize the active dependencies mapped between separate system segments. Objects must interact purely through abstract interfaces and contract definitions rather than rigid concrete types, preventing code changes from triggering a system-wide ripple effect.

5. Strategic Review Questions & Architectural Rationales

Question 1: How do classes differ from objects?

Rationale: A class functions as a static structural blueprint or declarative type metadata mapping common fields, operational methods, and semantic rules. An object represents a live, active instance of that blueprint occupying a distinct memory coordinate, owning its own independent runtime identity and active data states.

Question 2: How can software be designed to anticipate future porting to different platforms?

Rationale: This resilience is achieved by maintaining a strict separation between public interface boundaries and localized concrete implementations. Isolating platform-dependent dependencies (e.g., specific OS drivers or database dialects) behind abstract layers ensures the core business rules remain entirely clean and unimpacted during platform migrations.

Question 3: What happens if the separation between interface and implementation is not adhered to?

Rationale: The codebase falls into a highly brittle state known as tight coupling. Direct structural dependencies on concrete implementation code mean that even minor updates to a class's internal logic will force immediate, cascading modifications across all consuming modules, severely impacting automated unit testing and component reuse.

Question 4: Why is it recommended to avoid "data holder" only classes in OO design?

Rationale: Passive data holder entities strip away object intelligence and force behavior to accumulate inside large, external manager classes, dragging the system back into a procedural architecture pattern. Keeping actions paired directly alongside the attributes they modify honors the core software engineering principles of true encapsulation.